

AFLEWO Stack Decision — Config Packages, Cost, Prep & Gaps

1. Best Pick Per Category (Synthesized Recommendation)

Category	Pick	Why
Frontend hosting	Vercel Pro	Next.js-native, zero adapter risk, instant rollback — the free tier is the single biggest event-night liability in the whole stack
Database	Supabase Pro + Read Replica (add before events only)	You already have RLS + Auth wired in; ripping that out for PlanetScale/Neon costs more in migration risk than it saves
Auth	Stay on Supabase Auth	Already integrated with RLS. Add passkeys later as an enhancement, not a replacement
Media storage	R2 as source of truth, Cloudinary for transforms only	R2's zero-egress pricing is the only one that doesn't scale linearly with event-night photo uploads
CDN/Edge	Cloudflare (current \$5 plan, but write the WAF rules)	You're already paying for rate-limiting and DDoS protection you haven't turned on
Email	Brevo (keep)	Already integrated, free tier covers current volume
SMS	Africa's Talking	Only option with direct Safaricom/Airtel routing — generic providers (Twilio) route through international gateways with worse Kenya deliverability and cost
Observability	Sentry + BetterStack	Both have free tiers that fully cover AFLEWO's current scale — this is pure upside with no cost tradeoff
CI/CD	Vercel Git + one staging project + GitHub Actions smoke test	Minimum viable safety net without full DevOps overhead

2a) Total Pricing — Stack Configurations

Tier 1 — "Survive the Next Event" (minimum viable safety)

Item	Cost
Vercel Pro	\$20/mo
Supabase Pro + compute	\$34/mo
Cloudflare Workers Paid	\$5/mo
Sentry (free tier)	\$0
BetterStack (free tier)	\$0
Monthly total	\$59/mo
Labor (your hours, not cash)	~6–10 hrs: Vercel upgrade (5 min), DNS/orange-cloud (10 min), Sentry SDK install (30 min), BetterStack setup (15 min), WAF rules (20 min), k6 load test script + run (2–4 hrs), attendance QR flow if not built (variable, likely largest chunk)

Tier 2 — "12-Month Stable" (chapter growth, SMS fallback, staging)

Item	Cost
Everything in Tier 1	\$59/mo
Supabase Read Replica	+\$25/mo
Africa's Talking SMS (pay-per-use, est. 5k SMS/mo during event season)	~\$150/mo (event months only, ~\$0.03/SMS) — \$0 in off-months
Staging Vercel + Supabase project	~\$0-25/mo (can run staging on free tiers)
Monthly total (off-season)	~\$84-109/mo
Monthly total (event month)	~\$234/mo
Labor	Staging environment setup (~3–5 hrs), Africa's Talking integration (~4–8 hrs, new API + OTP flow), GitHub Actions smoke tests (~2–4 hrs)

Tier 3 — "3-5 Year Scale" (multi-country, mobile app, dedicated ops)

Item	Cost
Everything in Tier 2	~\$109/mo baseline
Supabase Team/Enterprise tier (multi-region conversations start here)	\$599+/mo (Enterprise pricing is quote-based — this is a floor estimate)
Dedicated mobile app (React Native/Expo) — one-time build	\$0 in cash if you build it, but 100+ hrs of dev time, or \$8k-20k+ if outsourced
Part-time technical/ops hire	\$300-800/mo (Kenya market rate for part-time ops-focused contractor)
Monthly total	\$1,000-1,500+/mo
Labor	This tier is the point where "labor" stops being your hours alone — it requires either significant personal time investment (200+ hrs across the transition) or delegating to a hire

Bottom line: Tier 1 is a 5x cost increase from your current \$39/mo but buys you out of the single biggest failure risk (Vercel free tier). Tier 2 only kicks in when SMS/staging actually matter. Tier 3 is a different conversation entirely — that's "AFLEWO becomes an institution with paid infrastructure staff," not an incremental upgrade.

2b) What to Prepare For

Languages/tools you already have covered (based on your track record across CEKA, Nasaka, SafeTrack, NOD):

- TypeScript/React/Next.js — no gap here
- Supabase (schema design, RLS, Edge Functions) — you've done this repeatedly, including fixing an infinite-recursion RLS bug this session
- Cloudflare (Workers, edge rendering) — you already resolved CEKA's SPA-indexability crisis using Cloudflare edge rendering

New tools/skills for this specific workload:

- **k6 (JavaScript-based load testing)** — you haven't load-tested at this scale before; k6 scripts are just JS, so the syntax isn't new, but the discipline of writing realistic traffic simulations is

- **SQL query performance reading (EXPLAIN ANALYZE)** — needed to actually verify PgBouncer pool sizing and catch slow JOINS before an event, not after
- **Sentry/BetterStack SDK config** — trivial technically (npm install + a few lines), but you need to decide what counts as "alert-worthy" so you don't get alert fatigue
- **Cloudflare WAF rule syntax** — declarative, not code-heavy, but you haven't written custom rules yet (only used the platform defaults)
- **Africa's Talking API** — new SDK, new auth flow (API key + username), Kenya-specific SMS/OTP patterns

Contexts you need to get literate in (not code, but decisions):

- **Kenya Data Protection Act 2019** — whether AFLEWO needs ODPC registration, what a compliant privacy notice looks like, right-to-erasure mechanics
 - **Incident response process** — this is a *process* gap, not a technical one: who gets paged, what the escalation path is, what "resolved" means
 - **Retention/lifecycle policy for media** — a decision you need to make once, not an ongoing skill
-

2c) Preparedness Assessment

Where you're genuinely ahead of a typical solo founder at this stage: Your history shows you've already handled several of the *hard* problems this audit is worried about, just in different projects: multi-Supabase-project management (CEKA + Nasaka + AFLEWO), a real production incident response (OTP rate limiting after an actual bot attack on CEKA), edge-rendering fixes under real SEO/traffic stakes, and a full crisis/continuity manual for CEKA ahead of a known high-traffic date (Finance Bill anniversary). That last one especially is directly transferable — you've done "prepare the system for a predictable high-stakes traffic day" before, just for a different kind of event.

Where the gap actually is: It's not coding ability — it's the *category* of problem. Everything you've built so far has been feature/product work with production hardening as a follow-up (fix the RLS bug after it breaks, add rate limiting after the bot attack happens). Event-night readiness is the opposite: it requires simulating the failure *before* it happens, because a live worship event doesn't get a hotfix window the way a website bug does. Load testing, capacity pre-scaling, and having monitoring live *before* the incident (not added after) is a different discipline than the iterate-and-fix loop you've been running successfully everywhere else.

Realistic read: You're probably 70% prepared on the technical execution (you could implement everything in Tier 1 within a weekend) and maybe 40% prepared on the operational discipline (load testing before rather than after, defined incident response, compliance paperwork) — because those aren't things your prior projects forced you to build yet.

3. Questions You're Missing (With Answers)

Q: Is AFLEWO a registered legal entity, separate from Saem's Tunes? This matters for liability if a data breach happens during an event with 7,000+ people's data. If AFLEWO is legally just "a project Saem runs," any DPA complaint or liability exposure may trace back to you personally rather than an organization. Worth clarifying with a lawyer, not just a technical decision.

Q: Who else has technical/admin access besides you? If you're the only person who can act during an incident, "who's on call" isn't really a question — it's always you. That's a single-point-of-failure risk worth naming explicitly, even if the honest answer right now is "just me."

Q: How does AFLEWO get billed — do you have a card that handles USD billing for Vercel/Supabase/Cloudflare without forex friction? This is a real, frequently underestimated cost for African founders: failed international card charges causing surprise service suspensions mid-event-season. Worth confirming your billing method is stable before scaling spend to Tier 2/3 levels.

Q: Is "AFLEWO" trademarked or brand-protected as it expands to more countries/chapters? Not a stack question, but relevant to "3–5 year scope" — if chapters in other countries start operating semi-independently, brand/name protection becomes a real gap.

Q: What happens if the venue itself loses internet, not your servers? Your stack review assumes the failure is server-side. But a church venue on a shared connection dropping to zero mid-event is arguably more likely than Vercel/Supabase going down. Is there an offline fallback for attendance (e.g., pre-generated QR codes cached on-device, paper backup)?

Q: Who owns the content rights to testimonies/photos submitted during events? Relevant to the Destroy Protocol conversation — if a member's testimony video needs to be taken down years later, do you have a documented consent/rights framework, or is it assumed implicitly?

Q: Is there a plan for what happens to AFLEWO's data/access if you personally become unavailable (illness, travel, etc.)? Given you're the sole technical owner across CEKA, Nasaka, SafeTrack, NOD, and Saem's Tunes simultaneously, this is a real bus-factor question worth a one-paragraph answer somewhere, even informally.

Want me to turn any one of these — the incident response runbook, the Kenya DPA compliance checklist, or the k6 load test script — into its own standalone deliverable next?